BSc (Hons) in Computer Science & Artificial Intelligence

Course Work: Critical Review and Technical Baseline of a Machine Learning

Research Paper

Reviewed Paper:

"FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning"

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024).

Publication: Proceedings of the AAAI Conference on Artificial Intelligence, 38(15), 16768-16776.

DOI: https://doi.org/10.1609/aaai.v38i15.29617

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030
- Module : Machine Learning Related Application
- Supervisor / Module Leader : Mr.Ashan Arthnayaka
- Date of Submission: Sep 15, 2026

Table of Contents

A. Problem Formulation..... 3
1.1 Introduction to the Problem..... 3
1.2 Core Machine-Learning Problem..... 3
1.3 Data Heterogeneity..... 4
1.4 Model Heterogeneity..... 4
1.5 Input Features..... 5
1.6 Target Variable..... 6
1.7 Learning Type..... 7
B. Methodological Justification..... 7
2.1 Introduction to the Methodology..... 7
2.2 Why the Authors Selected Trainable Global Prototypes..... 12
2.3 Why the Authors Selected ACL..... 13
2.4 Local Training Objective and Prediction..... 14
2.5 Client Models Used in FedTGP..... 15
2.6 FedTGP Methodology Diagram..... 16
C. Data Pipeline Analysis..... 18
3.1 Introduction..... 18
3.2 Overview of the Datasets..... 18
3.3 Preprocessing & Feature Scaling..... 21
3.4 Label Encoding..... 22
3.5 Class Imbalance and Client Data Distribution..... 23
3.6 Train-Test Splitting..... 24
3.7 Dataset Choice for Phase 2..... 24
References..... 25

Phase 1: Conceptual Mapping and Methodological Deconstruction

A. Problem Formulation

1.1 Introduction to the Problem

Traditional centralized machine learning involves users or organizations sending data to a single central server which is then used for model training. This method is effective but can result in privacy, security and data ownership issues, since the sensitive data has to be moved. It is also impractical where organizations are not able or willing to share their data directly [1], [2].

Federated Learning was introduced as an alternative approach. It allows several clients to train collaboratively while keeping their raw data on their own devices or organizations. Each client trains locally, and shares the learning information (model updates or class representations) selected by the client to the central server [2].

The selected paper, FedTGP, focuses on image classification in a heterogeneous federated-learning environment. The authors address a situation where clients have different data distributions and use different neural network architectures. FedTGP avoids directly sharing raw images and complete client-model parameters [1].

1.2 Core Machine-Learning Problem

The core problem is supervised multiclass image classification under data and model heterogeneity. Each client receives labelled image data and must learn to predict the correct image category. However, the clients do not have identical datasets or identical neural networks. This makes standard federated learning difficult because a single global model cannot be easily created by averaging their different model parameters.

For a client i, its local dataset can be represented as:

$$D_i = \{(x_{i,j}, y_{i,j})\}_{j=1}^{n_i}$$

Where $x_{i,j}$ is an input image, $y_{i,j}$ is its correct class label and n_i is the number of examples available on client i.

The final task is to learn a function that predicts the class label $\hat{y}$ for a new input image x [1].

1.3 Data Heterogeneity

Data heterogeneity means that different clients have different data distributions. In this paper, clients may have different numbers and types of images from each category.

For example, one CIFAR-10 client may mainly contain images of airplanes and automobiles, while another client may mainly contain cats and dogs. Therefore, the clients do not learn from the same type or proportion of data.

This can cause local models to become biased toward the classes that are common in their own data. A client trained mainly with vehicle images may be good at recognising vehicles but weak at recognising animals. The authors evaluate two non-IID data settings. In the pathological setting, every CIFAR-10 client receives examples from only two out of ten classes. In the practical setting, class proportions are distributed using a Dirichlet distribution with $\beta = 0.1$, which creates unequal class proportions across clients [1].

1.4 Model Heterogeneity

Model heterogeneity means that clients use different neural-network architectures.In FedTGP, the clients can use a small four-layer CNN, GoogLeNet, MobileNetV2, ResNet18, ResNet34, ResNet50, ResNet101 or ResNet152 [1]. It is realistic since various clients have various hardware, memory, processing and communication capability.

For example, a mobile device may be able to run only a small CNN or MobileNetV2, while a powerful organization may use ResNet50 or ResNet152.

The Ordinary Federated Averaging is unsuitable because of the model heterogeneity. FedAvg normally averages the parameters of local models, but the layers and parameter shapes of a CNN and ResNet50 are different. Their weights cannot be directly added or averaged. FedTGP addresses this issue by allowing every client to keep its own architecture. Instead of uploading complete models, clients create class prototypes in a shared 512-dimensional feature space and send those prototypes to the server [1].

1.5 Input Features

The input features are the RGB pixel intensity values of an image.

For CIFAR-10 and CIFAR-100, each image has a width of 32 pixels, a height of 32 pixels and three color channels: red, green and blue [3].

$$32 \times 32 \times 3 = 3,072$$

Therefore, each CIFAR image has 3,072 original pixel values.

In PyTorch format, the CIFAR input image is represented as:

$$x \in R^{3\times32\times32}$$

Tiny-ImageNet images have dimensions of $64 \times 64 \times 3$, while Flowers102 images are resized to 64×64 in the inspected generator [4], [5].

The original pixels are not directly shared between clients. Each local neural network extracts a learned feature vector from the image:

$$z_i = f_i(x; \theta_i)$$

Where f_i is client i's feature extractor, θ_i i represents its parameters and z_i is the learned feature vector. The paper uses a shared feature dimension of K = 512 by default [1].

1.6 Target Variable

The target variable is the image category label. It is categorical, not a continuous numerical value.

For CIFAR-10:

$$y \in \{0, 1, \ldots, 9\}$$

The possible targets are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck .

For CIFAR-100:

$$y \in \{0, 1, \ldots, 99\}$$

The target is one of 100 fine-grained object classes [3].

For Flowers102:

$$y \in \{0, 1, \ldots, 101\}$$

The target is one of 102 flower categories [6].

For Tiny-ImageNet:

$$y \in \{0, 1, \ldots, 199\}$$

The target is one of 200 object categories [4].

For the practical component of this coursework, CIFAR-10 will be used only. Therefore, the practical target variable will be the ten-class CIFAR-10 image label.

1.7 Learning Type

The task in FedTGP is supervised learning. Specifically, the task evaluated in FedTGP is supervised multiclass image classification. Federated Learning is a distributed training framework and can be used for supervised, unsupervised or reinforcement-learning tasks. However, the particular task studied by FedTGP is supervised because every training image has a corresponding class label [1].

B. Methodological Justification

2.1 Introduction to the Methodology

FedTGP was proposed because ordinary federated-learning methods struggle when clients have both **data heterogeneity** and **model heterogeneity**. The authors do not propose a new image-classification neural network. Instead, they propose a new **federated-learning method** that allows different neural-network models to collaborate [1].

The main idea is that clients do not send raw images or complete model weights to the server. Each client sends compact class representations called local prototypes. The server learns improved trainable global prototypes using Adaptive-Margin-Enhanced Contrastive Learning, or ACL [1].

The authors selected this approach after considering several earlier federated-learning methods. These methods are discussed below as the literature review and justification for FedTGP.

2.1.1 Federated Averaging: FedAvg

FedAvg is the original and most common federated-learning method. Each client trains a local neural network using its own data. It then sends the updated model parameters to a server. The server combines the parameters using weighted averaging [2].

$$\theta^{t+1} = \sum_{i \in S_t} \frac{n_i}{\Sigma_{j \in S_t} n_j} \theta_i^{t+1}$$

In this equation, S_t is the group of clients participating in round t, n_i is the number of samples on client i, and θ_i^{t+1} is its updated model.

FedAvg is appropriate when every client has the same neural-network architecture. For example, if all clients use ResNet18, the server can average corresponding layers and weights.However, FedTGP considers clients using different architectures, including a small CNN, MobileNetV2, GoogLeNet and several ResNet models. Their parameters have different shapes and numbers. Therefore, the authors cannot directly average the model weights.

This is the first reason why the authors did not select FedAvg as their main solution. Instead, they transfer **class-level knowledge** through prototypes [1].

2.1.2 Personalized Federated Learning

Personalized federated learning attempts to improve results when clients have different local data distributions. Instead of forcing every client to use exactly the same global model, personalized methods allow each client to maintain a model adapted to its own data [1].

These methods are helpful for data heterogeneity. For example a client with mostly animal images may need a slightly different model from a client with mostly vehicle images.However, FedTGP authors explain that personalized federated-learning methods alone do not fully solve model heterogeneity. Many still assume compatible model structures or a shared global model framework.

Therefore, the FedTGP authors needed an approach that could support both:

- different local data distributions; and
- genuinely different client architectures

FedTGP uses prototypes because the internal structures of client models can differ, while their outputs can still be mapped into a common feature space [1].

2.1.3 HeteroFL and FjORD

HeteroFL and FjORD are heterogeneous federated-learning approaches designed for clients with different computational resources [7], [8].

HeteroFL allows weaker clients to train smaller parts of a global model, while stronger clients train larger versions. FjORD uses ordered dropout so that clients can train different-sized submodels.These methods are useful when clients have different memory or computing power. However, the clients still use submodels derived from a common overall architecture. In other words, the models are different in size but are still structurally related.

FedTGP is more flexible because it allows clients to use different architecture families. For example, one client can use MobileNetV2 while another uses ResNet50. The authors choose prototype communication because it does not require all clients to share a common model design [1].

2.1.4 LG-FedAvg and FedGen

LG-FedAvg separates a client model into local and global parts. The lower layers can remain local, while upper layers are shared or aggregated. FedGen uses a generator to transfer knowledge between heterogeneous client models.These approaches are important because they also attempt to support model heterogeneity. However, the FedTGP authors identify limitations [1], [9].

LG-FedAvg still requires some compatible layers to be shared. This creates an architectural dependency. FedGen uses a generator, but its performance depends on whether the generator can produce useful information.In addition, statistical heterogeneity can affect the quality of information that is aggregated or generated. If clients have highly different data distributions, shared layers or generated knowledge may not represent every client equally well.

FedTGP avoids sharing model layers. It shares prototypes instead. A prototype is only a feature vector representing one class, so it is easier to exchange between different architectures [1].

2.1.5 Knowledge-Distillation Approaches

Knowledge distillation transfers knowledge from one model to another using predictions, probabilities or logits rather than model weights. In federated learning, some knowledge-distillation methods use a public or global dataset [1].

Clients make predictions for the same public examples, and these predictions are used to transfer knowledge across models.

This can support heterogeneity because different architectures can produce class probabilities for the same input. However, the FedTGP authors identify an important limitation: a suitable public dataset may not be available in real applications, especially where the client data is private or specialized [1].

2.1.5.1 FedDistill

FedDistill shares class-wise logits from clients. A logit is the numerical score produced by a classifier before it is converted into probabilities. The server aggregates the class information and returns it to clients to guide training [10].

FedDistill reduces the need to exchange complete model parameters. However, it shares knowledge at the output-prediction level. The FedTGP authors instead choose feature-level prototypes because they can describe class representations before the final classifier output.

2.1.5.2 FML and FedKD

Federated Mutual Learning or FML, and FedKD use auxiliary models and knowledge distillation to transfer information between heterogeneous clients. The auxiliary models or their compressed updates are compressed updates that are communicated to the server, creating higher

communication costs. During early training, both the private and auxiliary models may also have weak feature-extraction capabilities, which can reduce the quality of transferred knowledge [1].

2.1.6 FedProto and Prototype-Based Learning

FedProto is the most important previous method for understanding FedTGP. FedProto allows clients with different models to collaborate by exchanging class prototypes rather than complete model parameters [11].

A local prototype for class c on client i is calculated as:

$$P_i^c = \frac{1}{|D_{i,c}|} \sum_{(x,y) \in D_{i,c}} f_i\left(x; \theta_i\right)$$

Here, $D_{i,c}$ is the set of local images belonging to class c, and $f_i\left(x; \theta_i\right)$ is the feature vector extracted from image x.

For example, a client calculated the average feature vector of all its local cat images. This becomes its local cat prototype.

FedProto aggregates local class prototypes to create global prototypes. These global prototypes guide later local learning.

However, the FedTGP authors identify a weakness in simple prototype averaging. They explain that averaging can make the prototypes of different classes less separated. The authors describe this reduction in inter-class separation as prototype-margin shrinkage.

For example, if local cat and dog prototypes are positioned differently across clients, averaging may move the global cat and dog prototypes closer together. This can make it harder for clients to distinguish between the two categories.

The authors also argue that a client with more data does not always have a better prototype, Therefore, giving greater weight only because a client has more samples may not always create

the most meaningful global prototype. This limitation is the direct research gap addressed by FedTGP [1].

2.1.7 FedPCL

FedPCL also uses prototypes and contrastive learning. It uses pretrained models and applies contrastive learning on clients to improve their representation space.FedPCL is relevant because it shows that contrastive learning can improve prototype-based federated learning. However, the FedTGP authors state that FedPCL depends on pretrained models. In a real federated-learning environment, clients may have private architectures or limited data, making suitable pretrained models difficult to obtain [12].

FedTGP does not require clients to start with pretrained models. Instead, it learns global prototypes at the server from local prototype information [1].

2.2 Why the Authors Selected Trainable Global Prototypes

Trainable prototypes had previously been used in centralized machine learning to make representations from the same class more compact and to improve the separation between different classes. For domain adaptation, transferring knowledge between domains, similar approaches were also applied. However, these approaches normally require access to centralized data, model parameters or individual feature representations. Such access is unsuitable for federated learning, where clients must keep their data and models private [1].

FedTGP is an adaptation of trainable prototype learning for the federated setting. The basic prototype averaging, as used by FedProto, does not directly optimize the quality or separation of global prototypes. As a result, FedTGP generates a trainable vector for each class and processes this vector using a server-side prototype generator:

$$\widehat{P}^{c} = F\left(v^{c}; \theta_{F}\right)$$

Where:

- $\widehat{P}^c$ is the generated global prototype for class c;
- v^c is a trainable vector associated with class c;
- F is the server-side prototype generator;
- θ_F represents the server model parameters.

The Server-side prototype generator contains two fully connected layers with a ReLU activation function between them.This design enables the server to update and enhance global prototypes, instead of maintaining static averages. The server performs this learning using only the local class prototypes uploaded by participating clients. It does not access their raw images, individual feature representations or complete model parameters. In this way, client-model training and global-prototype learning are two parallel yet cooperative training processes throughout the rounds of communication [1].

2.3 Why the Authors Selected ACL

The authors use Adaptive-Margin-Enhanced Contrastive Learning (ACL) to train the global prototypes.The purpose of ACL is to make prototypes of the same class closer together and prototypes of different classes more separate [1].

The distance between two vectors is calculated using Euclidean distance:

$$\phi(a, b) = ||a - b||_2$$

The adaptive margin used in the paper is:

$$\delta_t = min\,(max_{c \neq ć}\, \phi(Q_t^c, Q_t^{ć}), \tau)$$

Where:

- δ_t is the adaptive margin.
- τ is the maximum allowed margin;
- Q_t^c is the average client prototype for class c at round t;

The margin is adaptive because it changes during training according to the current arrangement of class prototypes. It is limited by τ to avoid becoming excessively large.

The ACL objective for one local prototype is:

$$l_{i,c} = -\log \frac{e^{-[d(P_i^c, \widehat{P}^c)+\delta_t]}}{e^{-[d(P_i^c, \widehat{P}^c)+\delta_t]} + \Sigma_{\acute{c}\neq c} e^{-d(P_i^c, \widehat{P}^{\acute{c}})}}$$

In simple terms, this objective teaches the server that:

- A local cat prototype should be close to the global cat prototype;
- It should be far from global dog, horse and truck prototypes;
- The separation requirement should adapt as training progresses.

The authors selected ACL because it directly addresses the prototype-margin-shrinkage problem. ACL is intended to create global prototypes that are more discriminative than those produced by simple averaging [1].

The paper reports that FedTGP outperformed FedProto by up to 18.96% in its experiments Under the tested heterogeneous settings. This is the authors' reported experimental result, not a guarantee that FedTGP will outperform every method in every dataset or real-world situation [1].

2.4 Local Training Objective and Prediction

Each client trains with a combination of classification loss and prototype-alignment loss [1]:

$$L_i = L_{classification} + \lambda L_{alignment}$$

The classification loss encourages the local model to predict correct image labels. The alignment loss encourages local class representations to become close to the corresponding global prototypes.

The paper uses:

$$\lambda = 0.1$$

as the default alignment weight .

For prediction, FedTGP compares an image representation with all global prototypes:

$$\hat{y} = arg\ min_c \phi(f_i(x; \theta_i), \hat{P}^c)$$

The predicted class is the one with the closest global prototype [1].

2.5 Client Models Used in FedTGP

The paper's main heterogeneous model group is named HtFE8. It contains a four-layer CNN, GoogLeNet, MobileNetV2, ResNet18, ResNet34, ResNet50, ResNet101 and ResNet152 [1].

2.5.1 Four-layer CNN

The four-layer cNN is a relatively simple convolutional neural network. CNNs are designed for images because convolution layers learn local patterns such as edges, colors, textures and shapes. This model represents a client with limited computational capability [1].

2.5.2 GoogLeNet

GoogLeNet uses Inception modules.These modules process the same input through different branches, including different convolution sizes and pooling operations. It is included because it extracts image features differently from basic CNNs and ResNet models [1].

2.5.3 MobileNetV2

MobileNetV2 is a lightweight and efficient neural network architecture. It uses inverted residual structures and linear bottleneck layers. It represents devices with limited processing power, such as mobile or edge devices [1].

2.5.4 ResNet18

ResNet is a relatively shallow residual network. Residual connections help networks train effectively by allowing layers to learn changes relative to their inputs. It represents a moderate-complexity client model [1].

2.5.5 ResNet34

ResNet34 is deeper than ResNet18. It has a greater ability to learn complex visual patterns but it requires more computation [1].

2.5.6 ResNet50

ResNet50 is a deeper residual architecture using bottleneck blocks. It represents a more powerful client model [1].

2.5.7 ResNet101

ResNet101 is substantially deeper than ResNet50. It represents a high-capacity feature extractor [1].

2.5.8 ResNet152

ResNet152 is the deepest ResNet in the main HtFE8 group. It creates a strong contrast with the small CNN and MobileNetV2 [1].

The purpose of these models is not to decide which architecture is best. Their purpose is to create a difficult heterogeneous environment. FedTGP tests whether these different models can collaborate through shared 512-dimensional prototypes.

2.6 FedTGP Methodology Diagram

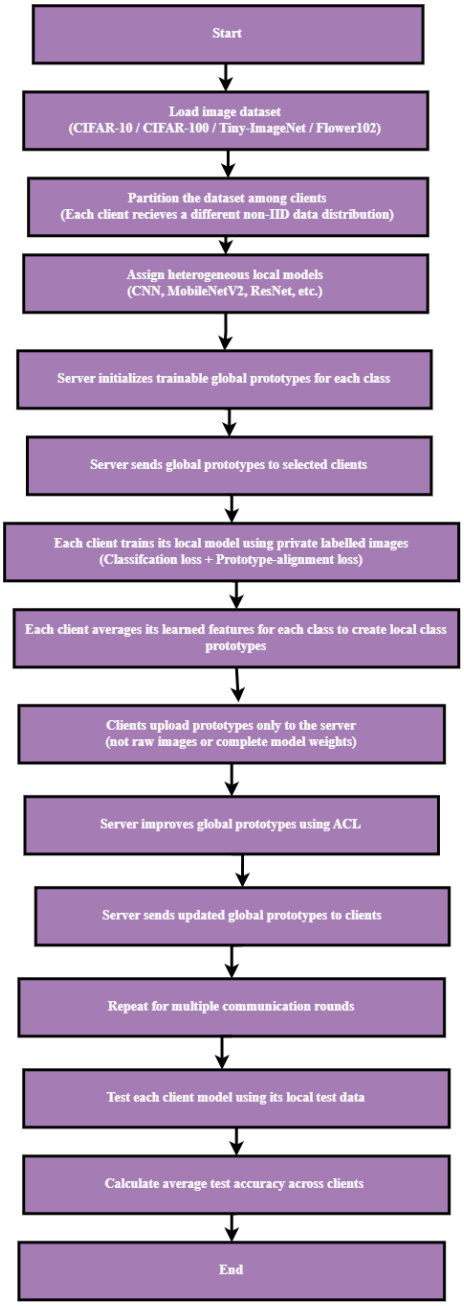


Figure 1. *Overall FedTGP training process* [1].

Figure 1 shows how FedTGP allows clients with different datasets and neural network models to collaborate. Clients train locally and upload only class prototypes; the server improves the global prototypes using ACL and sends them back for the next training round [1].

C. Data Pipeline Analysis

3.1 Introduction

FedTGP method is evaluated on four supervised image classification datasets: CIFAR-10, CIFAR-100, Flowers102 and Tiny-ImageNet [1]. These datasets differ in image size, number of classes, class balance and classification difficulty.

The data pipeline is crucial because FedTGP is not a single model trained with centrally collected data. Rather, the images are spread across 20 federated clients. The authors set up non-IID distributions where different classes and different class proportions are assigned to different clients. Thus, image preprocessing, client-level partitioning, local train/test splitting and treatment of class imbalance are also components of the pipeline.

3.2 Overview of the Datasets

Table 1. Overview of the datasets used to evaluate FedTGP [1], [3], [4], [6]

Dataset	Task	Number Classes	Input Size	Approximate number of images	Target Variable
CIFAR-10	Object classification	10	$32 \times 32\ RGB$	60,000	One of 10 object labels
CIFAR-100	Fine-grained object classification	100	$32 \times 32\ RGB$	60,000	One of 100 fine class labels
Flowers102	Flower-species classification	102	Variable RGB images	~8,000	One of 102 flower labels
Tiny-Image Net	Object classification	200	$64 \times 64\ RGB$	100,000 training images	One of 200 object labels

3.2.1 CIFAR-10 Dataset

CIFAR-10 is an image-classification dataset of 60,000 color images. It has ten classes such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. There are 6,000 images per class. The original training and testing sets include 50,000 and 10,000 images respectively [3].

Each image is 32×32 pixels with three color channels:

$$32 \times 32 \times 3 = 3,072$$

Therefore, the original input features are 3,072 RGB pixel intensity values. The target variable is one integer label:

$$y \in \{0, 1, \ldots, 9\}$$

The CIFAR-10 dataset is balanced at global level since there are an equal number of examples in each class. But once the data is distributed by the paper, it becomes locally imbalanced [1].

3.2.2 CIFAR-100 Dataset

CIFAR-100 contains 60,000 RGB images. Like CIFAR-10, each image is 32×32 pixels. However, CIFAR-100 has 100 fine-grained object classes rather than ten classes. Every class contains 600 images, with 500 images in the original training set and 100 in the original test set [3].

The input image contains:

$$32 \times 32 \times 3 = 3,072$$

pixel values. The target variable is one of 100 fine-grained object categories:

$$y \in \{0, 1, \ldots, 99\}$$

CIFAR-100 is more difficult than CIFAR-10 because the model must distinguish between 100 categories using small images. The dataset also contains 20 broader superclass categories, but the FedTGP experiment uses the 100 fine-grained categories as targets [1], [3] .

3.2.3 Flowers102 Dataset

Flowers102 is an image-classification dataset created by the University of Oxford. It contains approximately 8,000 flower images from 102 categories.Unlike CIFAR datasets, Flower102 is not balanced. Each category contains between 40 and 258 images. This creates natural class imbalance even before the data are divided among federated clients.The original images have different dimensions, lighting conditions, poses, backgrounds and flower appearances. Therefore, this dataset is more visually variable than CIFAR-10 [6].

The target variable is the flower category:

$$y \in \{0, 1, \ldots, 101\}$$

The dataset is useful for evaluating whether FedTGP can transfer class-level knowledge when the images are visually varied and the original class distribution is unequal.

3.2.4 Tiny-ImageNet dataset

Tiny-ImageNet is an image classification dataset based on ImageNet, but downsampled. According to the FedTGP paper, it is made up of 100,000 images with 200 classes [1]. Images are 64×64 pixels with three RGB channels .

Each image has dimensions:

$$64 \times 64 \times 3 = 12,288e$$

pixel values.

The target variable is one of 200 object categories:

$$y \in \{0, 1, \ldots, 199\}$$

Tiny-ImageNet is more difficult than CIFAR-10 because it contains more classes, larger images, and greater visual diversity. The higher number of classes also makes the non-IID client-distribution problem more challenging [1], [4].

3.3 Preprocessing & Feature Scaling

The FedTGP paper reports the experimental datasets and federated settings but does not list every transformation. Therefore, the following preprocessing details were obtained from the PFLlib dataset-generation code linked by the official FedTGP repository. The repository also refers users to HtFLlib for the required environment , and heterogeneous federated-learning resources [5], [13], [14].

For CIFAR-10 and CIFAR-100, the linked generators use $To\ Tensor()$ to convert each image into a PyTorch tensor. This also converts the pixel values from the original range of 0-255 to values approximately between 0 and 1. The generators then apply Normalize((0.5,0.5,0.5), (0.5,0.5,0.5)) to the three RGB channels [5].

For each colour channel, this transformation can be expressed as:

$$x_{normalized,k} = \frac{x_k - 0.5}{0.5} = 2x_k - 1$$

Where x_k is the converted tensor value of the pixel. As a result, the pixel values are mapped approximately from $([0, 1])$to $([-1, 1])$.

First, the generator associated with the Flowers102 is used to resize the image to 64×64 and then it is converted to a tensor and normalized. Resizing is a must since the dimensions of the original flower images vary, and the images that are in a training batch need to be the same size [5].

The Tiny-ImageNet generator also incorporates tensor conversion and normalization. The inspected generation scripts do not include additional random transformations, such as random

cropping, rotation or horizontal flipping. Thus, in this step the pipeline does not explicitly use training data augmentation, but rather transforms and scales it [5].

Normalization is useful because the neural network is sensitive to the magnitude of the features it is presented with. Keeping pixel values within a smaller and consistent range can support more stable gradient updates and more efficient model training.

A shared feature dimension must also be generated by FedTGP for all the heterogeneous client architectures. After each feature extractor the paper claims to add an average-pooling layer, and sets the default feature dimension to:

$$K = 512$$

This is not ordinary pixel scaling, but it is essential because the server must compare prototypes from CNN, GoogLeNet, MobileNetV2 and ResNet architectures [1].

3.4 Label Encoding

All four datasets use integer-encoded class labels.

CIFAR-10 uses IDs from 0 to 9. CIFAR-100 uses 0 to 99. Flowers102 uses 0 to 101. Tiny-Imagenet uses 0 to 199. The linked pipeline loads these integer labels directly, without putting them into one-hot encoding [3], [4], [5], [6].

Integer labels are suitable for a multiclass cross-entropy loss in PyTorch. All clients must have the same class map. Otherwise, a class label could have different meanings on different clients, resulting in incorrect local and global prototypes. There is no specific imputation of missing values. This is reasonable because the inputs are established image datasets rather than tabular records containing empty numerical cells. The pipeline uses the official dataset loaders to download or read the images and labels. But it does not have an individual procedure for missing images and corrupted files [5].

3.5 Class Imbalance and Client Data Distribution

The paper primarily studies statistical heterogeneity through non-IID level distributions, which produce severe client-level class imbalance [1]. It does not use SMOTE, oversampling or synthetic data generation.The paper uses two distribution settings [1].

3.5.1 Pathological non-IID setting

In the pathological setting, each client receives examples from only a limited number of classes.

For CIFAR-10, each client receives only two classes from the ten available classes. For CIFAR-100, each client receives ten classes. For Flowers102, each client receives ten classes. For Tiny-ImageNet, each client receives twenty classes.This creates severe data heterogeneity because a client may never see many classes during local training [1].

3.5.2 Practical non-IID setting

In the practical setting, the authors use a Dirichlet distribution:

$$q_c = (q_{c,1,\ldots,} q_c, M) \sim Dirichlet(\beta 1_M),$$

With

$$\beta = 0.1$$

A low value of β produces highly uneven class proportions. For example, one client may receive many examples of class “truck” while another client receives only a few or none [1].

CIFAR-10 and CIFAR-100 are balanced globally, but their local client datasets become imbalanced after this distribution. Flowers102 has both original class imbalance and additional client-level imbalance.

The authors use prototype-based knowledge sharing to address learning under these difficult data conditions. They do not claim to remove imbalance from the data.

3.6 Train-Test Splitting

After the data have been partitioned among clients, each client's private data are divided into:

75% training data

and

25% testing data

The linked implementation uses *train_test_split* with *train_size = 0.75* and shuffling enabled. The training portion updates the client's local model, while the test portion evaluates the model using unseen local samples [1], [5].

The parameter *stratify* is not directly used by the splitting function. Hence, it does not ensure that all the classes appearing in a client will be proportionally represented in both the training and test sets. This is especially true if the client has just a few samples in a class [5].

The same client partitions and local train/test samples should be applied to all methods employed in a practical comparison. If not, variations in performance might be due to different data splits and not to variation in learning algorithms.

3.7 Dataset Choice for Phase 2

The authors considered FedTGP on four datasets but only CIFAR-10 will be used in the phase 2 baseline implementation. This decision is made so that the practical work is feasible because federated training over multiple clients and communication rounds has substantial computational cost.

References

[1] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 15, pp. 16768–16776, Mar. 2024, doi: 10.1609/aaai.v38i15.29617.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," *Proc. 20th Int. Conf. Artif. Intell. Stat. PMLR*, vol. 54, latest revision 2023v4 2017, doi: 10.48550/arXiv.1602.05629.

[3] "CIFAR-10 and CIFAR-100 datasets." Accessed: Sep. 14, 2026. [Online]. Available: https://cave.cs.toronto.edu/kriz/cifar.html

[4] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "A Downsampled Variant of ImageNet as an Alternative to the CIFAR Datasets," 2017, *arXiv*. doi: 10.48550/ARXIV.1707.08819.

[5] TsingZ0, *TsingZ0/PFLlib*. (Sep. 12, 2026). Python. Accessed: Sep. 14, 2026. [Online]. Available: https://github.com/TsingZ0/PFLlib

[6] M.-E. Nilsback and A. Zisserman, "Automated Flower Classification over a Large Number of Classes," in *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, Bhubaneswar, India: IEEE, Dec. 2008, pp. 722–729. doi: 10.1109/ICVGIP.2008.47.

[7] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients," 2020, *arXiv*. doi: 10.48550/ARXIV.2010.01264.

[8] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. I. Venieris, and N. D. Lane, "FjORD: Fair and Accurate Federated Learning under heterogeneous targets with Ordered Dropout," 2021, *arXiv*. doi: 10.48550/ARXIV.2102.13451.

[9] Z. Zhu, J. Hong, and J. Zhou, "Data-Free Knowledge Distillation for Heterogeneous Federated Learning," 2021, doi: 10.48550/ARXIV.2105.10056.

[10] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-Efficient On-Device Machine Learning: Federated Distillation and Augmentation under Non-IID Private Data," 2018, *arXiv*. doi: 10.48550/ARXIV.1811.11479.

[11] Y. Tan *et al.*, "FedProto: Federated Prototype Learning across Heterogeneous Clients," *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, pp. 8432–8440, Jun. 2022, doi: 10.1609/aaai.v36i8.20819.

[12] Y. Tan, G. Long, J. Ma, L. Liu, T. Zhou, and J. Jiang, “Federated Learning from Pre-Trained Models: A Contrastive Learning Approach,” 2022, *arXiv*. doi: 10.48550/ARXIV.2209.10083.

[13] TsingZ0, *TsingZ0/FedTGP*. (Jul. 27, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/FedTGP

[14] TsingZ0, *TsingZ0/HtFLlib*. (Sep. 08, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/HtFLlib